Important Point

- friend function breaks encapsulation.
- It must be declared in all classes whose private data it access.
- It can be a global function or even a member of another class.

Real life Analogy

Think of a class as a private room, only member can enter.

A friend function = special guest without permission.

Problem statement: Sum of private data using friend fn

- two classes
- int a of first and b of 2nd
- add both a and b.

Examples

```
#include <iostream>
class Complex {
    int a,b ;
    public :
        void setNumber (int N1, int N2) {
            a = N1;
            b = N2;
        }

    friend complex sumComplex(complex O1, complex O2);

    void printNumbers () {
    cout << "your number is " << a << "+" << b << "i"
    << endl;
    }
};

complex sumcomplex (complex O1, complex O2) {
    complex O3 ;
    O3. setNumber ((O1.a + O2.a) , (O1.b + O2.b));
    return O3 ;
}

int main () {
    complex C1, C2, Sum ;
    C1.setNumber (1,4) ;
    C1. printNumber () ;

    C2. setNumber (5,8)
    C2. printNumber ( ) ;

    Sum = sumComplex( C1, C2) ;
    Sum. printNumber ( ) ;
}
```

```
s[i].showdata();
Checkresult(s[i]);
    if (s[i].marks >= highest)
    {
        highest = s[i].marks;
    }
}

cout << "highest marks : " << highest << endl;
return 0;
}.
```

#Day-18 (Friend function)

A friend function is a function that is not a member of a class, but it can still access private and protected member of that class.

Normally private data cannot be accessed outside the class. But with friend you allow access

Why to use it?

- when two or more class need to shared data
- when a function need direct access to private mem
- helps in operator overloading (future topic)

Syntax

Syn

```
class class_name {
    friend return_type function_name(arguments);
};
```